

Újrahasznosított mobiltelefonokon alapuló biztonsági rendszer megvalósítása

SwarmCam

Vass Richárd · 26223071

Mentor: dr. Pletl Szilveszter

Szabadka, 2026. szeptember

A probléma és a cél

A probléma

- **Felhőalapú konzumer kamerák** — előfizetéshez kötöttek, és a felvétel elhagyja a házat: a működés egy külső szolgáltató rendelkezésre állásán múlik.
- **Dedikált IP kamera + NVR** — külön hardverkötség, és kameránként kézi konfiguráció a rögzítőben.
- **Közben a hardver adott** — a világ éves e-hulladék-termelése az 50 millió tonnás nagyságrendben mozog. Egy 3–4 éves telefonban ott a kameramodul, a hardveres H.264 enkóder, a Wi-Fi rádió és az akkumulátor — jellemzően nem meghibásodás miatt cserélik le.

A CÉL

Felhőfüggetlen, kizárólag helyi hálózaton működő, mesterséges intelligenciával támogatott kamerarendszer, amely újrahasznosított Android telefonokat integrál — dedikált kamerahardver nélkül.

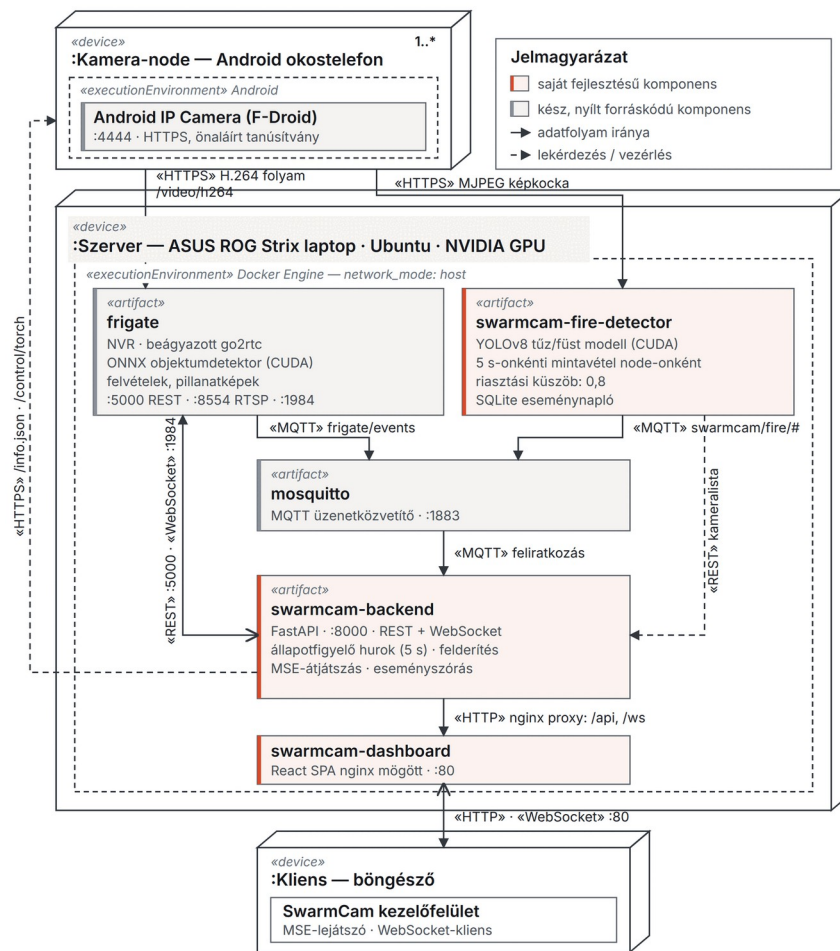
Internetkapcsolat nélkül is teljes értékű.

HÁROM TERVEZÉSI KÉNYSZER

- A node nem ONVIF-eszköz → egyedi ujjlenyomat-vizsgálat kell.
- A node bármikor eltűnhet → minden komponensnek túlnie kell a forrás hiányát.
- A detekció a limitáló tényező → minden felesleges kódolási lépés ki kell essen.

3.2. SZAKASZ

A SwarmCam architektúrája



— **Kamera-node** — Android IP Camera app: nyers H.264 folyam, JSON állapotleíró és távoli beállítás. Tetszőleges számú (1..*).

— **go2rtc → Frigate** — a go2rtc veszi át a telefon folyamatát és helyi RTSP-ként szolgálja ki újra; a Frigate ezt dekódolja, detektál és rögzít.

— **Mosquitto** — két, egymást nem ismerő eseményforrás (Frigate, tűzdetektor) közös gyűjtőpontja.

— **FastAPI backend** — állapotfigyelés, eseményátjátszás, élőkép-relay, Frigate-proxy.

— **React dashboard** — egyoldalas felület, asztali és mobil elrendezéssel.

— **Tűzdetektor** — a rétegsoron kívül: csak a brókerhez csatlakozik.

Az 1. ábrán pirossal jelölt artefaktumok saját fejlesztésűek.

Mi a saját munkám, és mi a készen kapott

SAJÁT TERVEZÉS ÉS MEGVALÓSÍTÁS

- hálózati felderítés és a hozzá tartozó ujjlenyomat-vizsgálat
- FastAPI backend és valós idejű rétege — állapotfigyelő hurok, MQTT-átjátszás, WebSocket-elosztás
- élőkép-architektúra a go2rtc MSE-relayjével
- a kameraforgatás feldolgozási lánc
- a tűz-/füstdetekciós mikroszolgáltatás teljes Python-pipeline-ja
- React kezelőfelület, asztali és mobil elrendezéssel
- a 3. fejezet tervezési döntései és a 3.6. szakasz mérési módszertana

KÉSZEN KAPOTT KOMPONENSEK

- Frigate NVR és a benne futó go2rtc
- Android IP Camera alkalmazás (F-Droid)
- Mosquitto MQTT bróker
- előre betanított YOLO tűz-/füstmodell — ezt nem én tanítottam be, csak a saját pipeline-omba integráltam

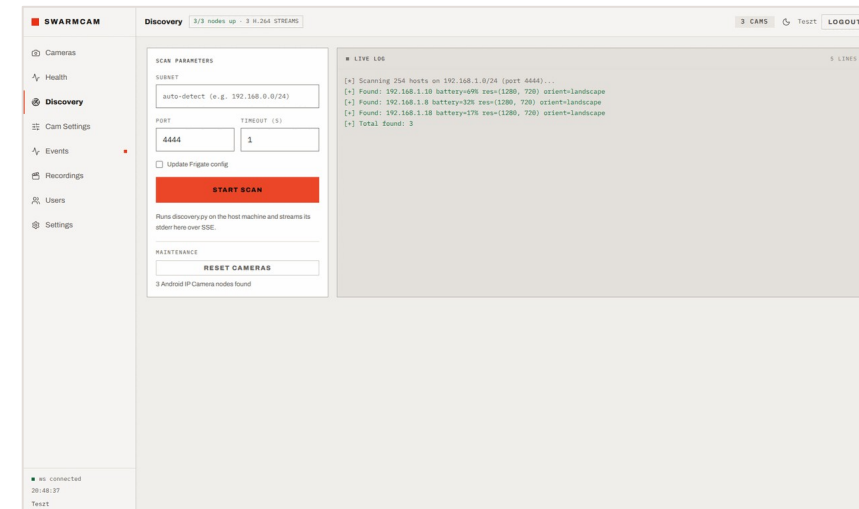
Saját detekciós motort tudatosan nem írtam: a valós idejű, többkamerás objektumdetekció — mozgásmaszkkal, objektumkövetéssel, hardveres gyorsítással — önmagában kitöltene egy szakdolgozatot, és gyengébb eredményt adna a meglévő nyílt forráskódú megoldásnál. A munka súlypontja a köré épített integrációs réteg.

Automatikus hálózati felderítés

— **Kétlépcsős ujjlenyomat.** Előbb gyors TCP-portpásztázás a 4444-es porton, 128 párhuzamos szálon, 1 s időkorláttal — egy /24-es alhálózat 254 címe néhány másodperc. Utána HTTPS-en az /info.json leíró, és az ujjlenyomat a válasz szerkezete: az Android IP Camera mindig egyszerre adja vissza a cameras és a settings kulcsot.

— **Determinisztikus névadás.** cam_ + az IP-cím, pontok helyett aláhúzással — cam_192_168_1_8. Ugyanaz a telefon újraindítás után is ugyanazt az azonosítót kapja, így a felvételei és beállításai nem szakadnak el tőle. Ez a névséma a teljes kameraprotokoll-cserét is túlélte, migráció nélkül.

— **Önálló szkript, SSE-vel.** A backend külön folyamatként indítja: a több száz párhuzamos socket nem terheli az event loopot, a kimenete pedig Server-Sent Events csatornán élőben követhető.



EGY TANULSÁGOS HIBA

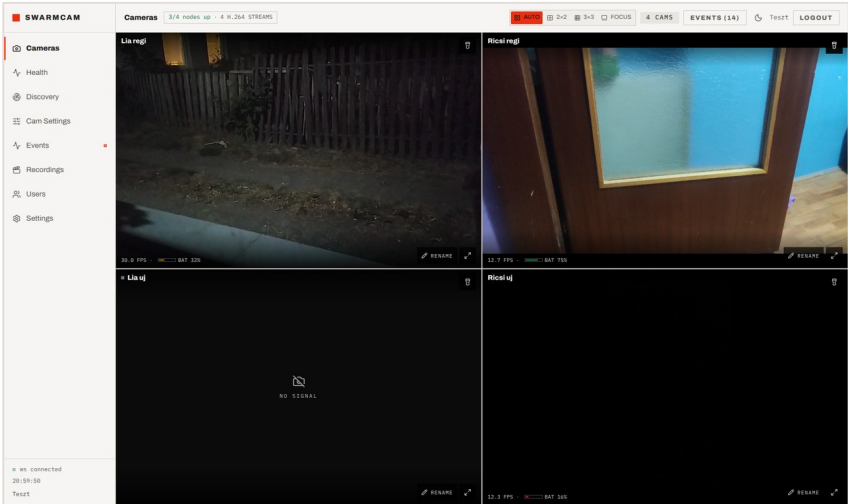
A leíró auto vagy max kulcsszóval is megadhatja a folyam méretét. Az első megvalósítás ilyenkor a szenzor legnagyobb támogatott méretét vette — ám az a legnagyobb fényképméret, nem a videófolyamé. A szám a Frigate detekciós méretébe került, és mivel a Frigate a mozgásdetektálást teljes detekciós felbontáson végzi, egyetlen kamera közel 300% processzoridőt evett. A javítás: bizonytalan méret esetén nem írunk detekciós méretet.

Élőkép: MJPEG-proxytól a go2rtc MSE-ig

- **Az első megoldás** a Frigate hibakeresési MJPEG-folyamát proxyzta. Működött, de három szerkezeti baja volt: a detekciós képkockákat kódolta JPEG-be (640×360, 5 fps), nézőnként külön processzorterhelést jelentett, és összecsatolta az élőkép minőségét a detekció beállításaiával.
- **A mostani megoldás** a go2rtc MSE-csatornája: a telefon H.264 folyamát fMP4 szegmensekké csomagolja át — újrakódolás nélkül —, és WebSocketen adja. A backend ezt játssza át; a böngésző a natív H.264 dekóderével jeleníti meg.
- **A relay szándékosan minimális:** két, egymással szemben futó továbbító feladat. Az élőkép-útvonalon a backend egyetlen videóműveletet sem végez — nincs dekódolás, átkódolás és pufferek.

11. TÁBLÁZAT — EGY NÉZŐ, UGYANAZ A RENDSZER

Szempon	MJPEG-proxy	go2rtc MSE
Megjelenített felbontás	640×360	1280×720
Képfrissítés	3,5 kép/s	a telefon natív fps-e
Frigate CPU / néző	+26 pp	+2 pp
Késleltetés	nem mérve	0,03–0,07 s
Szerveroldali újrakódolás	JPEG nézőnként	nincs



Push, nem lekérdezés: MQTT → WebSocket

- **Minden csatorna arra a feladatra került, amelyre való.** REST ott, ahol a kliens tudja, mikor van szüksége az adatra — kameranista, események, felvételek. WebSocket ott, ahol a szerver kezdeményez: kamerastátusz, detektált események, riasztások, mindez egyetlen tartós kapcsolaton.
- **A mérés szándékosan szintetikus eseményekkel dolgozik.** A Frigate üzenetformátumával megegyező eseményeket publikáltam a brókerre, így a mért érték kizárólag a rendszer saját átjátszási láncát jellemzi — bróker, backend, WebSocket-továbbítás —, és nem tartalmazza a Frigate detektálási idejét. Azt a 3.6.3. szakasz méri külön.
- **Az alternatíva mérlege.** Egy másodperces periódusú lekérdezés ugyanezt átlagosan félmásodpercre növelte volna, ráadásul üresjáratú terhelés árán. A mért nagyságrend a felületen érzékelhetetlen.

0,7 ms

átlagos átjátszási késleltetés
MQTT-től a böngészőig

1,2 ms

95. percentilis

29 / 30

illesztett eseménypár
120 s mérés alatt

1 db

illesztetlen esemény —
a mérési ablak határán

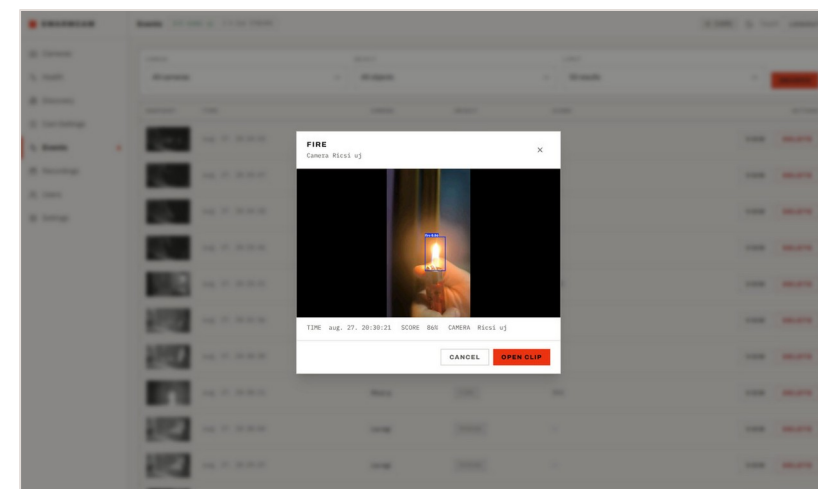
Tűz- és füstérzékelés külön mikroszolgáltatásként

— **A feladatkiírás előírta** a rendszer kiegészítését egy olyan eseményosztállyal, amelyet az alapul szolgáló NVR általános detektora nem támogat. A tűz és a füst a Frigate COCO-80 címkekészletében nem szerepel.

— **Miért nem a Frigate modelljének cseréje?** A Frigate egyszerre egy modellt futtat, tehát a tűzfelismerő betöltése az általános objektumdetekció elvesztésével járna. Két feladat két modellt kíván; párhuzamos futtatásukra a külön folyamat a legegyszerűbb mód.

— **A működés egy egyszerű ciklus.** A szolgáltatás lekéri a kameralistát a backendtől, a telefon MJPEG-folyamából kiolvassa az első teljes JPEG-képkockát, és azon YOLO-modellt futtat a szerver GPU-ján. Kameránként 5 másodpercenként egy képkocka — rövid, ritka lökésekben terheli a GPU-t, így a Frigate detektorával érdemben nem versenyez.

— **A bróker az egyetlen csatolási pontja.** Nem hívja a backendet, nem ír közös adatbázisba, és a felület létezéséről sem tud — csak publikál a `swarmcam/fire/<kamera>` témára. Ezért a Frigate leállása mellett is működik.



Hogyan mértem

- **A nehézség.** A kameraszám és a detektor összehasonlító méréséhez minden mérési pontban azonos bemenet kell, különben a pontok különbsége nem a vizsgált változóból ered. Élő telefonokkal ez nem biztosítható: a jelenet, a megvilágítás és így a képsebesség percről percre változik.
- **A megoldás.** A bemenet egy korábban, ugyanezekről a telefonokról rögzített, valós H.264 felvétel — 1280×720, 10 kép/s —, amelyet a Frigate minden mérési pontban ugyanúgy, N különálló kameraként dolgoz fel.
- **A mérőszkript.** Mérési pontonként 60 másodpercen át, 2 másodpercenként mintavételezi a konténerek erőforrás-használatát; a pontok között a rendszer 60 másodpercet stabilizálódik.
- **A haszon.** Minden mérési pont pontosan azonos bemenetet kap, a mérés pedig bármikor megismételhető — éjjel-nappal, a telefonok állapotától függetlenül.

7. TÁBLÁZAT — A MÉRÉSI KÖRNYEZET

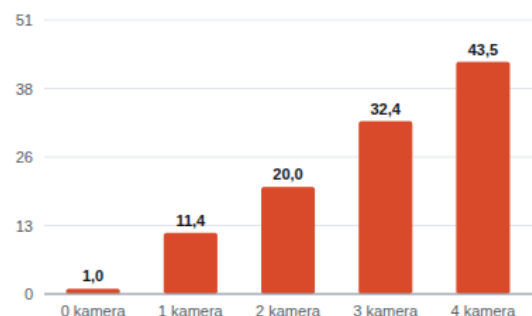
Elem	Érték
Processzor	Ryzen 9 5900HX, 8/16
Memória	15 GiB
GPU	RTX 3060 Laptop, 6 GiB
OS	Ubuntu 24.04.3 LTS
Konténer	Docker 28.5.2
Frigate	0.17.2 stable-tensorrt
Detektor	ONNX Runtime, yolo-generic
Bemenet	320×320, COCO-80
Kamera-node	1280×720 H.264, HTTPS

Az egyetlen eltérés az éles üzemtől, hogy itt a Frigate közvetlenül olvassa a felvételt, a go2rtc újracsomagoló lépése nélkül. A go2rtc nem kódol újra, csak konténerformátumot vált — a hozzájárulása kicsi és állandó.

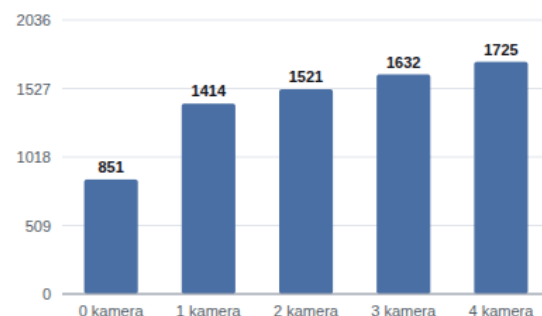
Erőforrás-használat a kameraszám függvényében

Erőforrás-használat a kameraszám függvényében

FRIGATE — PROCESSZORHASZNÁLAT (%)



FRIGATE — MEMÓRIA (MiB)



— **Az alapterhelés elhanyagolható.** Kamerák nélkül a Frigate processzorhasználata 1% alatt marad; ami mérhető, az szinte teljes egészében memória — a betöltött modell és a futtatókörnyezet önmagában 851 MiB.

— **A processzorterhelés lineárisan nő.** 11,4% → 20,0% → 32,4% → 43,5%; a növekmény kameránként ~10,6 százalékpont. A rendszer erőforrásigénye tehát előre becsülhető, és a mérési tartományban nincs torlódást okozó közös erőforrás.

— **A többi konténer független a kameraszámtól.** A backend minden mérési pontban 1% CPU alatt marad, 48–52 MiB memóriával. Az erőforrásigény egyetlen komponensen, a videófeldolgozáson múlik.

— **Mérlegben.** A 43,5% a docker stats konvenciója szerint egy magra vonatkozik — négy kamera a 16 szás gép kapacitásának 2,7%-a. A szűk keresztmetszet nem a processzor, hanem a detektor átbocsátóképessége és a lemez írási sávszélessége.

Detektálási sebesség: GPU-gyorsítás és processzor

43,12 ms → 5,54 ms

következtetési idő képkockánként, azonos
modellel
és azonos bemenettel — 7,8× gyorsulás

23,2 → 180,5 det/s

elméleti detekciós felső korlát:
a detektor egyetlen, sorosított erőforrás (1000/n)

14,07 vs 15,68 kép/s

3 kamera × 5 fps = 15 det/s igény —
a processzoros futtatás épp itt telítődik

- **A processzoros futtatás nem használható** — csak a kiszolgálható kameraszámot korlátozza. A mért értékekből a processzor 4–5 kameránál telítődne, míg a GPU ugyanezen a beállításon több mint harminc kamerát kiszolgálna.
 - **A mérőgép processzora 16 szál, tehát viszonylag erős.** Egy tipikus, alacsony fogyasztású mini PC-n a processzoros érték lényegesen rosszabb lenne — ez a GPU, vagy alternatívaként egy Coral típusú célhardver melletti érvet tovább erősíti.
 - **A mérést megismételtem** egy másik felvételrészletből épített bemenettel: 7,4 vs 36,4 ms, azaz 4,9-szeres arány. Azon a bemeneten lényegesen kevesebb mozgás volt, a detektor nem telítődött, így a detektálásszám nem jellemezte a kapacitást. A táblázat ezért a telített futás adatait közli; mindkét futás nyers adata megtalálható az elektronikus mellékletben.
 - **A tisztaság feltétele.** A két futtatás mindenben megegyezik — azonos modell, bemenet, kameraszám és detekciós felbontás —, kizárólag a detektor futtatási eszköze tér el. A mérés idejére a mozgásküszöb a lehető legalacsonyabbra került, hogy a detektor gyakorlatilag minden képkockán lefusson.
- Az alapértelmezett konfiguráció ennek ellenére szándékosan processzoros detektorral indul — az minden gépen működik. A GPU-gyorsítás a beállítások felületén, egyetlen legördülő menüből kapcsolható be.**

A képsebesség fényfüggése — a telefonoldali korlát



- **Nem a szerver, nem a hálózat és nem a beállított érték.** Az élőkép képsebességét a kamera előtti megvilágítás szabja meg.
- **Az ok fizikai.** Az Android automatikus expozíciója gyenge fényben megnöveli az expozíciós időt. Ha egy képkocka rögzítése 300 ms-ig tart, másodpercenként legfeljebb három képkocka állítható elő — függetlenül attól, mit kér a szerver.
- **Reprodukálható.** Ugyanaz a node, változatlan jelenet és szerveroldali beállítások: sötétben 3,3 kép/s, a zseblámpa bekapcsolása után 11,3 kép/s.
- **A következtetés.** Ez szerveroldalról nem kezelhető, mert a képkockák elő sem állnak. Aki telefonalapú kamerarendszert épít, annak a megvilágításról külön kell gondoskodnia — éjszakai üzemben ez a legfontosabb gyakorlati korlát.
- **Ezért mutat a felület mért értéket,** nem beállítottat: az 25 kép/s-ot sugallna, miközben három van.

Megfigyelt hibamódok és a küszöbhangolás

- ① **A node eltűnik a hálózatról.** A rendszer szempontjából nem hiba, hanem üzemszerű állapot: az állapotfigyelő hurok néhány másodpercen belül észleli, a felület egyértelműen jelzi, a többi node zavartalanul működik tovább.
- ② **A telefon videókódolója elakad.** A készülék hálózatilag elérhető marad — az állapotleíró válaszol, a node „online” —, a H.264 végpont viszont üres választ ad. A jelenlegi állapotfigyelés ezt nem ismeri fel, mert az elérhetőséget az állapotleíróval méri, nem a videófolyammal; a felület közvetve mégis jelzi, mert a kártya képsebessége nullára esik. Ez a legfontosabb nyitott pont.
- ③ **Téves tűzriasztás gyenge fényben.** A modell éjszakai, erősen zajos képeken füstöt jelez ott, ahol nincs — a nagy erősítéssel készült kép szenzoraja statisztikailag hasonlít arra, amit a modell füstként tanult meg.

A KÜSZÖB HANGOLÁSA — MÉRÉSI TAPASZTALAT, NEM ALAPÉRTTELMEZÉS

A detektor eseménynaplójában rögzített találatok átnézése azt mutatta, hogy az eredeti 50% és 80% közötti sávba eső detektálások lényegében kivétel nélkül tévesek voltak, míg a valódi lángra adott találatok pontszáma — a mért esetekben 82 és 86 százalék között — ezt a sávot meghaladta. A küszöb 80%-ra emelése után a téves éjszakai riasztások megszűntek, valódi találat elvesztése nélkül.

Ez a küszöb a jelen telepítés kameráira illeszkedik, általános érvényűnek nem tekinthető; a robusztusabb megoldás a képfényesség alapján váltó, külön éjszakai küszöb volna.

Korlátok és a következő lépések

TUDATOSAN VÁLLALT KORLÁTOK

- **Megbízható helyi hálózat.** Névtelen MQTT, ellenőrzés nélkül elfogadott ön aláírt tanúsítvány, host hálózati névtér, Docker-socket. Internet felőli kitétség esetén mindegyik önálló sebezhetőséggé válna — távoli eléréshez VPN a helyes megoldás, nem portátírányítás.
- **Az állapot a memóriában él.** A kameralista induláskor a Frigate konfigurációjából épül újra. Egy csomópontra és kis kameraszámmra megfelelő, de kizárja a többcsomópontos üzemet.
- **A node fekete doboz.** A protokollcserével elveszett a töltési állapot jelzése, távoli újraindításra nincs mód. Nem megvalósítási hiba, hanem annak a döntésnek a következménye, hogy a node-oldalt készen kaptam.
- **A tűzmodell nem saját betanítású.** Nyilvánosan elérhető súlykészlet, tehát nem erre a telepítésre optimalizált.

HA MA KEZDENÉM / KÖVETKEZŐ LÉPÉSEK

- **Saját kamera-alkalmazás.** Nyers H.264 helyett szabványos RTSP-kiszolgáló, helyes időbélyegeekkel, és a forgatás már a kódolás előtt, a telefon oldalán. Ezzel a szerveroldali újrakódolás teljes problémaköre — a processzorköltség és az időbélyeg-vesztés — meg sem jelenne.
- **A videófolyamot is figyelő health-check,** hogy a beragadt kódoló ne mutakozzon „online”-nak.
- **Fényviszony-érzékeny tűzküszöb,** illetve transfer learning saját gyűjtésű adathalmazon.
- **Coral TPU kiértékelése** alacsonyabb fogyasztású telepítésekhez; ONVIF/PTZ kamerák bevonása; a telefon további szenzorai (mikrofon, gyorsulásmérő) mint új eseményforrás — az MQTT-alapú, szétcsatolt architektúra ezt átalakítás nélkül fogadni tudná.

Összegzés

- **A kitűzött részcélok megvalósultak:** automatikus felderítés, felhőfüggetlen szerverkörnyezet, AI-alapú objektum- és tűz-/füstdetekció, valós idejű eseménykezelés és egységes kezelőfelület. A moduláris felépítés — különálló konténerek, jól definiált MQTT/WebSocket interfészek — lehetővé teszi az egyes komponensek önálló továbbfejlesztését.
- **A felderítés azt a lépést váltja ki,** amely a hagyományos NVR-eknél kameránként kézi munka. De ez nem konfigurációmentes üzem: a node-oldali beállítás — felbontás, képsebesség és különösen a tájolás — megmarad. A rendszer a kamerák bekötésének kézi konfigurációját szünteti meg, nem a telepítés egészét.

HÁROM TANULSÁG

A kész node fekete doboz marad.

Nem én döntöm el, milyen adat érhető el róla és milyen formában — ez nem hiba, hanem a „készen kapom” döntés ára.

Minden újrakódoló réteg rejtett költséget hoz,

és nem ott, ahol keressük: a forgatás az időbélyegeken keresztül a detekciót állította le, miközben a felvétel zavartalanul készült.

A mérés a tervezés része,

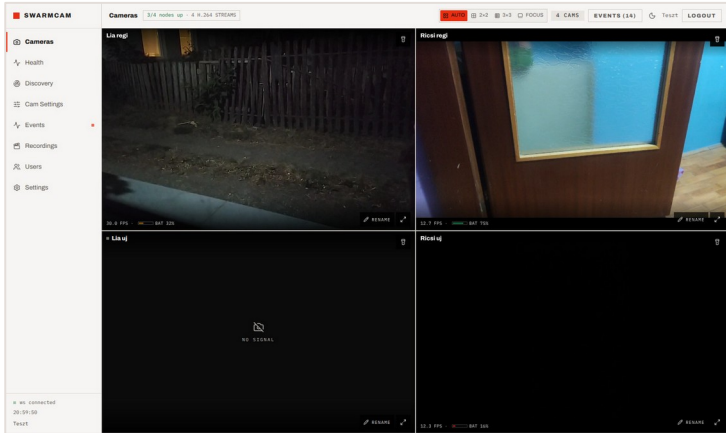
nem az utólagos igazolásé. Az élőkép cseréje, a detektor GPU-ra terelése és a tűzküszöb megválasztása szemre eldönthetetlen döntés volt.

Tartalék diák

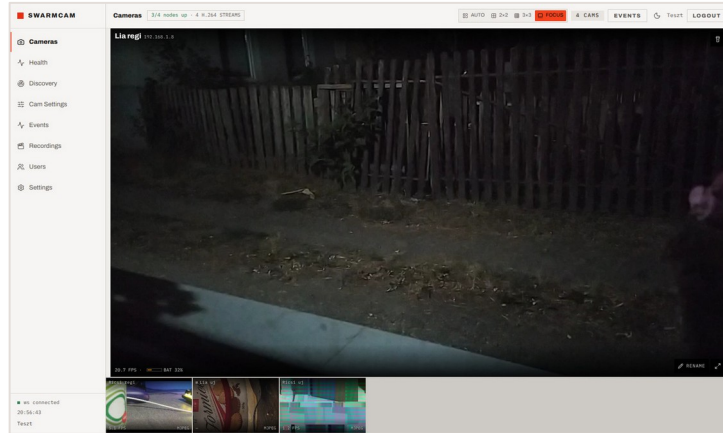
Tartalék diák

a bizottság kérdéseihez

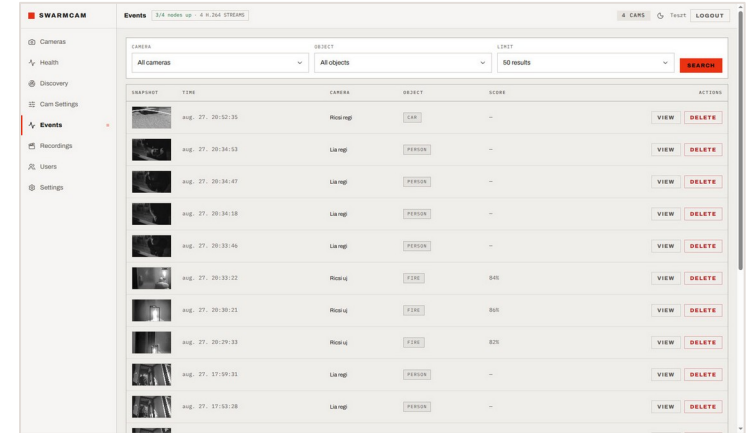
Tartalék · Kezelőfelület — asztali elrendezés



Rácsnézet — élőkép MSE-n, node-onkénti mért fps, akku és Wi-Fi

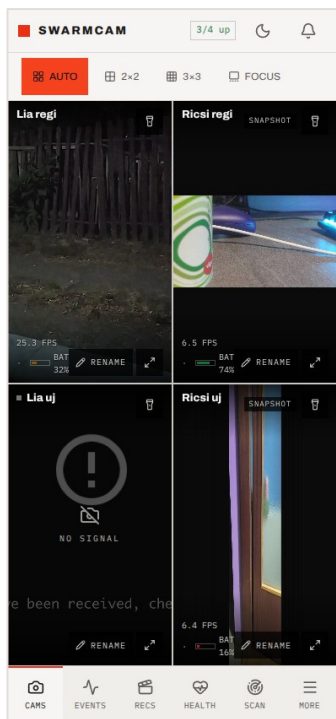


Fókusz elrendezés — egy kamera nagyban, a többi az alsó sávban

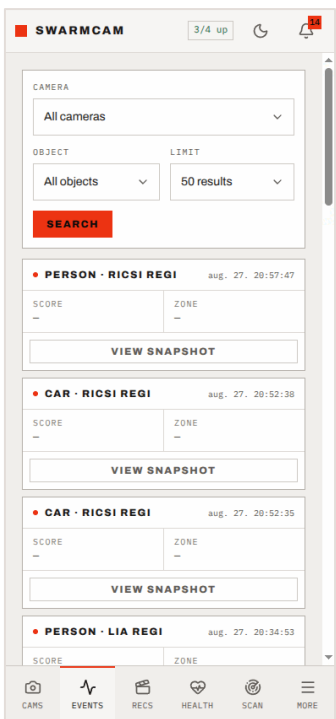


Eseménybongészó — a Frigate detektált eseményei, klippel

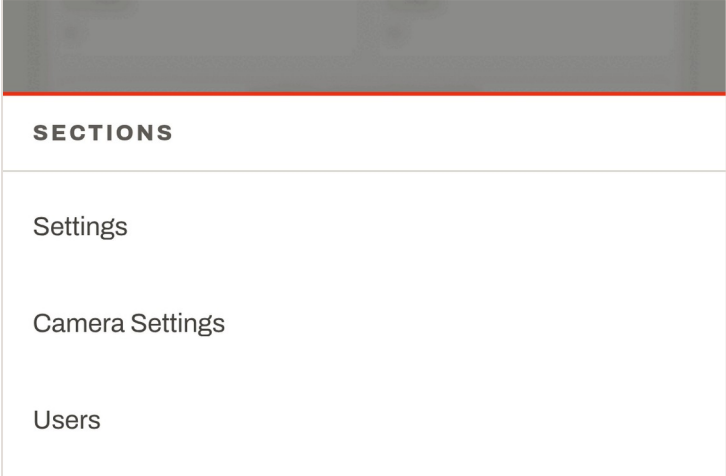
Tartalék · Kezelőfelület — mobil elrendezés



Kamerarács — az oldalsáv helyét rögzített fejléc és alsó fülsáv veszi át



Az asztali táblázat kártyás nézeté alakul



A fülsávba nem férő nézetek alulról felcsúszó lapon

Tartalék · 8. táblázat — erőforrás-használat kameraszámonként

Kameraszám	0	1	2	3	4
frigate — CPU (%)	1,0	11,4	20,0	32,4	43,5
frigate — memória (MiB)	851	1414	1521	1632	1725
backend — CPU (%)	0,40	0,56	0,45	0,61	0,55
backend — memória (MiB)	48	49	50	52	52
fire-detector — CPU (%)	0,40	0,27	0,36	0,33	0,32
fire-detector — memória (MiB)	131	131	131	131	131
mosquitto — CPU (%)	0,04	0,04	0,03	0,03	0,03
dashboard — CPU (%)	0,00	0,01	0,01	0,01	0,01

60 másodperc alatt, 2 másodpercenként vett minták átlaga. A docker stats konvenciója szerint a százalék egyetlen processzormaghoz viszonyul.

Tartalék · 9. táblázat — a detektor következtetési ideje

Mutató	GPU (ONNX, CUDA)	Processzor (ONNX, CPU)
Következtetési idő képkockánként (ms)	5,54	43,12
Legnagyobb mért következtetési idő (ms)	5,57	47,07
Összesített detekciós képsebesség (kép/s)	15,68	14,07
Elméleti detekciós felső korlát (kép/s)	180,5	23,2

Azonos modell, azonos bemenet, 3 kamera, telített detektor. A megismételt mérés (másik felvételrészlet): GPU 7,4 ms, processzor 36,4 ms — 4,9-szeres arány, mert azon a bemeneten a detektor nem telítődött, így a detektálásszám nem jellemezte a kapacitást.

Tartalék · 4. táblázat — a kommunikációs csatornák és indoklásuk

Csatorna	Szereplők	Adat	A választás indoka
REST / HTTP	frontend ↔ backend ↔ Frigate	kameralista, események, felvételek, konfiguráció	szinkron, állapotmentes erőforrás-hozzáférés
WebSocket	backend → frontend	kamerastátusz, események, riasztások	a szerver kezdeményez; egy tartós kapcsolat mindenre
WebSocket (bináris)	go2rtc → backend → böngésző	fMP4 videószegmensek	a meglévő nginx- és hitelesítési útvonalon halad; nincs ICE és UDP-port
MQTT	Frigate és tűzdetektor → bróker → backend	detekciós események	szétcsatolt közzététel: a források nem ismerik egymást

Tartalék · Hitelesítés és a biztonsági modell határa

- **A hitelesítés a Frigate felhasználói adatbázisára épül.** A backend továbbítja a bejelentkezési adatokat, és a kapott JWT-t adja vissza a felületnek; minden további kérés ezt hordozza.
- **A token kizárólag a React alkalmazás memóriájában él,** nem a böngésző tartós tárolójában — az ugyanis minden, az oldalon futó szkript számára olvasható lenne, tehát egy sikeres szkriptbeillesztés azonnal átvehető munkamenetet adna. Ára, hogy a token az oldal újratöltésekor elvész; helyi adminisztrációs felületen ez elfogadható.
- **Két független munkamenet.** A backend saját hitelesített munkamenetet is fenntart a Frigate felé, mert az állapotfigyelés, a statisztikák és a konfiguráció olvasása felhasználói beavatkozás nélkül, folyamatosan zajlik.
- **401 ≠ 503.** A 401 — érvénytelen vagy lejárt token — kijelentkeztet; az 503 — a Frigate épp újraindul egy konfigurációmentés után — nem. A kettő összemosása azt jelentené, hogy minden konfigurációmentés kijelentkeztet.
- **A modell határa.** A SwarmCam megbízható, helyi hálózatra készült. Az MQTT névtelen hozzáférést enged, az önaláírt tanúsítványt ellenőrzés nélkül elfogadjuk, a konténerek a gazdagép hálózati névterét használják, és a backend hozzáfér a Docker-sockethez. Internet felőli kitettség esetén mindegyik önálló sebezhetőség — a helyes megoldás VPN, nem portátirányítás.